

UC LINA 600 LINEAR ALGEBRA FOR GRAPHICS · MODULE 07 · LECTURE NOTES

Homogeneous Coordinates

A fourth number that makes translation linear.

MODULE OUTCOMES

- Explain why translation needs an extra dimension.
- Use homogeneous coordinates for points and directions.
- Build affine transform matrices.
- Explain the perspective divide.
- Distinguish points from directions correctly.

1 The problem

EVERY LINEAR MAP FIXES THE ORIGIN, SO TRANSLATION CANNOT BE A MATRIX — IN THREE DIMENSIONS

$T(0) = T(0 \cdot v) = 0 \cdot T(v) = 0$ — so a linear map always sends the origin to the origin (Module 03 §4), and translation manifestly does not. This is a proof, not an inconvenience.

Which would mean carrying every transform as a matrix *and* a separate vector, composing the two parts by different rules, forever — **workable, ugly, and error-prone**, and it is what you have to do in systems that lack this trick.

The trick: embed three-dimensional space as the plane $w = 1$ inside a four-dimensional space — and a shear in 4D, restricted to that plane, is exactly a translation in 3D.

So translation *is* a linear map, one dimension up — which is a genuinely elegant move rather than a bookkeeping convenience, and is why every graphics transform matrix is 4×4 even though the geometry is three-dimensional.

2 Points and directions

```
A POINT has w = 1.      (x, y, z, 1)
A DIRECTION has w = 0.  (x, y, z, 0)
```

```
AND THE ARITHMETIC THEN WORKS OUT CORRECTLY:
point - point      = direction  (1 - 1 = 0)
point + direction = point      (1 + 0 = 1)
direction + direction = direction
point + point      = w of 2 -- which is
                    meaningless, and the w
                    tells you so
```

```
WHY IT MATTERS FOR TRANSFORMS
the translation column of the matrix multiplies
w. A point (w=1) therefore gets translated. A
direction (w=0) does NOT -- which is correct,
since translating a direction is meaningless,
and the convention handles it automatically
with no special case.
```

```
SO LIGHT DIRECTIONS GET w = 0, and vertices get
w = 1, and translation needs no branch.
```

```
NORMALS ALSO TAKE w = 0, but that settles only
translation. Their 3x3 part still needs the
inverse transpose of Module 06 -- w = 0 does
not rescue a normal under non-uniform scale.
```

A direction has $w = 0$, so translation ignores it — which is correct and automatic, and is why the convention is worth following rather than storing everything as $w = 1$ and remembering which is which. **Storing a direction with $w = 1$ is a real bug and it translates your light vectors:** the lighting then changes as the object moves, which looks like an entirely different problem.

3 Affine transforms

THE 4×4 IS A 3×3 LINEAR PART, A TRANSLATION COLUMN, AND A BOTTOM ROW THAT IS USUALLY (0,0,0,1)

The top-left 3×3 block is rotation, scale, and shear — everything from Module 03, unchanged and composing as before.

The fourth column is the translation — which is where the object's position lives, and is the first thing to read when inspecting an unfamiliar matrix in a debugger: if the object is in the wrong place, look there first.

A bottom row of (0, 0, 0, 1) means the transform is affine — parallel lines stay parallel, and w is left alone — and anything else in that row makes it a projective transform, which is §4.

So you can read a 4×4 at a glance: the linear part, the position, and whether it is affine or projective — three things, directly from the layout, which makes matrix debugging far less opaque than it first appears.

4 The perspective divide

- **Perspective means dividing by distance** — something twice as far away appears half as large — and **division is not a linear operation**, so no matrix can perform it.
- **But a matrix can arrange for the depth to end up in the w component** — a bottom row of (0, 0, -1, 0) copies $-z$ into w — and the division then happens as a separate step afterwards, dividing x , y , and z by w .
- **So the projection matrix does not project: it sets up a division that the hardware performs in the next stage** — which is why that stage is called the perspective divide and is listed separately in every pipeline diagram.
- **Which is the piece of the pipeline people find most mysterious**, and it is **one row of one matrix plus one division**, which is considerably less than its reputation suggests.
- **And it explains the near clip plane: as z approaches zero, w approaches zero, and the divide explodes** — which is why there is a near plane at all, and why putting it very close to the camera destroys depth precision (Module 08 §3). **'The projection matrix does not project; it sets up a division' is the sentence that makes Module 08 straightforward.**

AND THIS IS THE IDEA THE WHOLE PIPELINE DEPENDS ON

One extra coordinate makes translation linear and makes perspective expressible — two separate problems solved by one trick, which is why it is worth understanding rather than accepting as a convention.

And it is the reason every transform matrix in graphics is 4×4 , including the ones that are doing nothing but rotating something in three dimensions.

Plus the practical rule: $w = 1$ for points, $w = 0$ for directions, and check it whenever something gets translated that should not have been — which is a two-second diagnosis for a class of bug that otherwise looks mysterious.

Which is Module 08's prerequisite — that module derives the projection matrix from scratch, and this is the entire machinery it needs.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Gortler, chapters 3 and 11 (library copy)	§§1 to 3 — points versus directions treated properly, which most texts blur.
Scratchapixel, the homogeneous coordinates article (free)	§§1 and 4 — free, with the perspective divide derived step by step.
Lengyel, chapter 4 (library copy)	§3 — the 4×4 structure in engine conventions.
Hartley & Zisserman, chapter 2 (library copy)	§4 rigorously — projective geometry properly, which is what CSCE 753 will want.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Prove that a linear map fixes the origin.**
- 2 **Show that no 3×3 matrix can translate.**
- 3 **Build a 4×4 translation matrix** and verify it on points.
- 4 **Apply it to a direction ($w = 0$)** and confirm nothing happens.
- 5 **Store a direction as $w = 1$ deliberately** and watch the lighting change as the object moves.
- 6 **Read three unfamiliar 4×4 matrices** and state the linear part, position, and type of each.

- 7 **Build a matrix that copies $-z$ into w .**
- 8 **Apply it and perform the divide by hand** on several points.
- 9 **Plot apparent size against distance** and confirm the reciprocal.
- 10 **Move the near plane very close** and observe what happens to depth precision.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why can translation not be a 3×3 matrix?
- 2 What is the trick, stated geometrically?
- 3 What do $w = 1$ and $w = 0$ mean?
- 4 Why does point minus point give a direction?
- 5 Why does translation ignore directions automatically?
- 6 Describe the four parts of a 4×4 affine matrix.
- 7 What does the bottom row tell you?
- 8 Why is perspective not a linear operation?
- 9 What does the projection matrix actually do?
- 10 Why is there a near clip plane?